Python & Algorithms for Coding Interviews

Section 3A — Core Algorithmic Patterns (3.1 – 3.10)

3.1 Two Pointers

The Problem Situation

You are given a sorted array (or string) and need to find a pair, triplet, or subarray satisfying some condition. A brute force nested loop would be $O(n^2)$. Two pointers reduce this to $O(n)$ by moving a left and right pointer intelligently toward each other.

Key Insight

Insight	When the array is sorted, you can place one pointer at the start and one at the end. If their combined value is too large, shrink from the right. If too small, grow from the left. Each step moves at least one pointer, so the total work is $O(n)$.

Analogy

Mental Model	Two people walking toward each other on a number line. If together they overshoot the target, the person on the right takes a step left. If they undershoot, the person on the left takes a step right. They never pass each other and they always converge in $O(n)$ steps.

Basic Code Template

```
# Two pointers — opposite ends (sorted array)
def two_sum_sorted(nums, target):
    left, right = 0, len(nums) - 1
    while left < right:
        total = nums[left] + nums[right]
        if total == target:
            return [left, right]
        elif total < target:
```

```
            left  += 1   # need larger sum
        else:
            right -= 1   # need smaller sum
    return []
```

✅**This Template Solves**	LeetCode 167 — Two Sum II • LeetCode 15 — 3Sum • LeetCode 11 — Container With Most Water

Alternative Template

```
# Two pointers — same direction (partition / remove)
def remove_duplicates(nums):
    if not nums: return 0
    slow = 0
    for fast in range(1, len(nums)):
        if nums[fast] != nums[slow]:
            slow += 1
            nums[slow] = nums[fast]
    return slow + 1
```

✅**This Template Solves**	LeetCode 26 — Remove Duplicates • LeetCode 27 — Remove Element • LeetCode 977 — Squares of Sorted Array

Example Walkthrough

```
# Example: LeetCode 15 — 3Sum
# Find all unique triplets that sum to 0
def three_sum(nums):
    nums.sort()
    result = []
    for i in range(len(nums) - 2):
        if i > 0 and nums[i] == nums[i-1]:  # skip duplicates
            continue
        left, right = i + 1, len(nums) - 1
        while left < right:
            total = nums[i] + nums[left] + nums[right]
            if total == 0:
                result.append([nums[i], nums[left], nums[right]])
                while left < right and nums[left]  == nums[left+1]:  left
+= 1
                while left < right and nums[right] == nums[right-1]: right
-= 1
                left += 1; right -= 1
            elif total < 0: left  += 1
            else:           right -= 1
    return result

# Trace: nums = [-4,-1,-1,0,1,2]
# i=1 (val=-1): left=2,right=5 → -1+(-1)+2=0 ✓ → append
# i=1 (val=-1): left=3,right=4 → -1+0+1=0  ✓ → append
```

Time & Space Complexity

Case	Time	Space
Typical	O(n)	O(1)
With sort	O(n log n)	O(1)

Pattern Recognition Clues

- Keywords: sorted array/string, pair/triplet, sum/difference target
- Input is sorted OR can be sorted first without changing the answer
- Problem asks for a pair or asks to partition an array in place
- $O(n^2)$ brute force is obvious — two pointers gives O(n)

Interview Traps

□ **Trap**	Forgetting to sort first — two pointers only work reliably on sorted arrays
□ **Trap**	Not skipping duplicate values in 3Sum — leads to duplicate triplets in result
□ **Trap**	Using left <= right instead of left < right — processes same element twice

Edge Cases

- Empty array or single element
- All elements are the same
- No valid pair exists — must return empty or -1
- Negative numbers in sum problems

Important Practice Problems

- LeetCode 167 — Two Sum II - Input Array Is Sorted
- LeetCode 15 — 3Sum
- LeetCode 11 — Container With Most Water
- LeetCode 42 — Trapping Rain Water
- LeetCode 977 — Squares of a Sorted Array

Real World Python Usage

□ **Real World**	Two pointers are used in merge operations inside Merge Sort. Database JOIN operations on sorted columns use a similar merge pointer technique. Network packet deduplication scans sorted packet IDs with two pointers.

3.2 Sliding Window

The Problem Situation

You need to find a subarray or substring of a specific size or satisfying a condition (max sum, longest without repeating, etc.). Brute force checks all $O(n^2)$ subarrays. Sliding window maintains a running window and expands or shrinks it in O(n) total.

Key Insight

□ **Insight**	Instead of recomputing from scratch for every subarray, maintain a window between two pointers. When the window becomes invalid (too large, has duplicates, etc.), shrink from the left. The total number of left and right moves is O(n).

Analogy

□ **Mental Model**	A magnifying glass sliding across a page. You see a fixed-width section at a time. To move right, you add the new right character and drop the old left character. You never restart from the beginning — you just slide.

Basic Code Template

```
# Fixed-size window — max sum of subarray of size k
def max_sum_fixed(nums, k):
    window_sum = sum(nums[:k])
    best = window_sum
    for i in range(k, len(nums)):
        window_sum += nums[i] - nums[i - k]  # add right, remove left
        best = max(best, window_sum)
    return best
```

✓**This Template Solves**	LeetCode 643 — Maximum Average Subarray I • LeetCode 1456 — Maximum Number of Vowels in Substring

Alternative Template

```
# Variable-size window — longest substring without repeating chars
def length_of_longest_substring(s):
    seen  = {}   # char → last seen index
    left  = 0
    best  = 0
    for right, ch in enumerate(s):
        if ch in seen and seen[ch] >= left:
            left = seen[ch] + 1   # shrink window
        seen[ch] = right
        best = max(best, right - left + 1)
    return best
```

✅This Template Solves	LeetCode 3 — Longest Substring Without Repeating • LeetCode 209 — Minimum Size Subarray Sum • LeetCode 567 — Permutation in String

Example Walkthrough

```
# Example: LeetCode 3
# s = 'abcabcbb'
# right=0 (a): seen={a:0}, window='a',      len=1
# right=1 (b): seen={a:0,b:1}, window='ab',  len=2
# right=2 (c): seen={..c:2}, window='abc',   len=3
# right=3 (a): a seen at 0 >= left(0) → left=1
#              seen={a:3}, window='bca',      len=3
# right=4 (b): b seen at 1 >= left(1) → left=2
#              window='cab', len=3
# Answer: 3
```

Time & Space Complexity

Case	Time	Space
Fixed window	O(n)	O(k) or O(alphabet)
Variable window	O(n)	O(k) or O(alphabet)

Pattern Recognition Clues

- Keywords: subarray, substring, contiguous, window, at most k, longest/shortest
- Problem involves a range of consecutive elements
- You need max/min/count of something within a range that changes
- Brute force is $O(n^2)$ — sliding window reduces to O(n)

Interview Traps

□ **Trap**	Resetting the entire window on invalidity instead of just shrinking from the left
□ **Trap**	Off-by-one in window size: window length = right - left + 1, not right - left
□ **Trap**	Forgetting to update seen index — must update even when character is already in window

Edge Cases

- Empty string or array
- k larger than the array length
- All characters the same
- Window that requires counting (use Counter or defaultdict)

Important Practice Problems

- LeetCode 3 — Longest Substring Without Repeating Characters
- LeetCode 76 — Minimum Window Substring
- LeetCode 209 — Minimum Size Subarray Sum
- LeetCode 567 — Permutation in String
- LeetCode 239 — Sliding Window Maximum

Real World Python Usage

□ **Real World**	Real-time streaming analytics systems compute moving averages using sliding windows. Network rate limiters track requests in a rolling time window. Log analysis systems find peak traffic windows. Video processing computes frame-level statistics over sliding temporal windows.

3.3 Prefix Sum

The Problem Situation

You need to answer multiple range sum queries on an array, or find a subarray whose sum equals a target. Naively computing each range sum is O(n) per query. Prefix sums allow O(1) range queries after O(n) preprocessing. Combined with a hash map, prefix sums solve subarray sum problems in O(n).

Key Insight

Insight	Precompute a running total. The sum of any subarray [i, j] = prefix[j+1] - prefix[i]. You convert a range sum query from O(n) work into a single subtraction. For finding subarrays with a given sum, store prefix sums in a hash map and look up the complement.

Analogy

Mental Model	A bank account running balance. Instead of adding up all transactions between two dates, you take the balance on the end date and subtract the balance on the start date. One subtraction gives you the total — no matter how many transactions are in between.

Basic Code Template

```
# 1D Prefix Sum — range query in O(1)
def build_prefix(nums):
    prefix = [0] * (len(nums) + 1)
    for i, num in enumerate(nums):
        prefix[i+1] = prefix[i] + num
    return prefix

# Sum of subarray [left, right] inclusive
def range_sum(prefix, left, right):
    return prefix[right+1] - prefix[left]

# Usage
nums   = [1, 2, 3, 4, 5]
prefix = build_prefix(nums)  # [0,1,3,6,10,15]
print(range_sum(prefix, 1, 3))  # sum of [2,3,4] = 9
```

✓This Template Solves	LeetCode 303 — Range Sum Query • LeetCode 304 — Range Sum Query 2D • LeetCode 1480 — Running Sum of 1D Array

Alternative Template

```
# Prefix Sum + Hash Map — subarray sum equals k
def subarray_sum(nums, k):
    count   = 0
    current = 0
    seen    = {0: 1}   # prefix_sum → frequency
    for num in nums:
        current += num
        # if (current - k) was seen before, a valid subarray ends here
        count += seen.get(current - k, 0)
        seen[current] = seen.get(current, 0) + 1
    return count
```

✅**This Template Solves**	LeetCode 560 — Subarray Sum Equals K • LeetCode 974 — Subarray Sums Divisible by K • LeetCode 525 — Contiguous Array

Example Walkthrough

```
# Example: nums=[1,2,3], k=3
# current=0, seen={0:1}
# num=1: current=1, need=1-3=-2, seen.get(-2)=0, seen={0:1,1:1}
# num=2: current=3, need=3-3=0,  seen.get(0)=1  → count=1
#        seen={0:1,1:1,3:1}
# num=3: current=6, need=6-3=3,  seen.get(3)=1  → count=2
# Answer: 2  (subarrays [1,2] and [3])
```

Time & Space Complexity

Case	Time	Space
Build prefix array	O(n)	O(n)
Range query	O(1)	O(1)
Subarray sum with hashmap	O(n)	O(n)

Pattern Recognition Clues

- Keywords: range sum, subarray sum, sum equals k, cumulative, running total
- Multiple queries on the same array — build prefix once, answer in O(1)
- Find subarray with exact sum — prefix sum + hash map
- 2D grid range sum — 2D prefix sum

Interview Traps

⬜ **Trap**	Off-by-one in indexing: prefix[i+1] = prefix[i] + nums[i] — the prefix array is size n+1
⬜ **Trap**	Forgetting to initialise seen = {0: 1} — misses subarrays that start from index 0
⬜ **Trap**	Using prefix sum on a problem that needs min/max — prefix sum only works for sum

Edge Cases

- Empty array

- All negative numbers
- k = 0 — subarrays that sum to zero
- Multiple subarrays with same prefix sum — count correctly with frequency map

Important Practice Problems

- LeetCode 303 — Range Sum Query - Immutable
- LeetCode 560 — Subarray Sum Equals K
- LeetCode 974 — Subarray Sums Divisible by K
- LeetCode 525 — Contiguous Array
- LeetCode 304 — Range Sum Query 2D - Immutable

Real World Python Usage

Real World	Database GROUP BY with cumulative SUM uses prefix sum logic. Financial systems compute running P&L using prefix sums. Image processing uses 2D prefix sums (integral images) for fast region-average calculations. Pandas .cumsum() is a direct prefix sum implementation.

3.4 Hash Map Counting

The Problem Situation

You need to count frequencies, find duplicates, detect anagrams, or find a value that satisfies a relationship with another value seen earlier (like Two Sum). Hash maps give O(1) lookup and let you trade space for time — turning $O(n^2)$ brute force into O(n).

Key Insight

Insight	Store what you have seen so far in a hash map. For each new element, check if the map already contains what you need. This is the classic 'store and look up' pattern — you answer the question 'have I seen the complement of this before?' in O(1).

Analogy

Mental Model	You are at a party looking for someone wearing the same colour shirt as you. Instead of asking every person ($O(n^2)$), you keep a list of shirt colours you have seen (O(n) space). For each new person you meet, you check your list in one second.

Basic Code Template

```
# Two Sum — classic hash map pattern
def two_sum(nums, target):
    seen = {}   # value → index
    for i, num in enumerate(nums):
        complement = target - num
        if complement in seen:
            return [seen[complement], i]
        seen[num] = i
    return []
```

✅**This Template Solves**	LeetCode 1 — Two Sum • LeetCode 167 — Two Sum II • LeetCode 653 — Two Sum IV BST

Alternative Template

```
# Frequency map — group anagrams
from collections import defaultdict

def group_anagrams(words):
    groups = defaultdict(list)
    for word in words:
        key = tuple(sorted(word))   # anagrams share same sorted key
        groups[key].append(word)
    return list(groups.values())

# Top K frequent elements
from collections import Counter
def top_k_frequent(nums, k):
    freq = Counter(nums)
    return [x for x, _ in freq.most_common(k)]
```

✅**This Template Solves**	LeetCode 49 — Group Anagrams • LeetCode 347 — Top K Frequent Elements • LeetCode 242 — Valid Anagram

Example Walkthrough

```
# Example: Two Sum — nums=[2,7,11,15], target=9
# i=0 (2):  complement=7,  not in seen → seen={2:0}
# i=1 (7):  complement=2,  2 in seen at index 0 → return [0,1]

# Example: Group Anagrams — ['eat','tea','tan','ate','nat','bat']
# 'eat' → sorted → ('a','e','t') → groups[('a','e','t')]=['eat']
# 'tea' → sorted → ('a','e','t') → groups[('a','e','t')]=['eat','tea']
# 'tan' → sorted → ('a','n','t') → groups[('a','n','t')]=['tan']
# ...
# Result: [['eat','tea','ate'], ['tan','nat'], ['bat']]
```

Time & Space Complexity

Case	Time	Space
Single pass	O(n)	O(n)
Frequency count	O(n)	O(k) unique elements

Pattern Recognition Clues

- Keywords: two sum, frequency, count, duplicate, anagram, most common
- You need to find a complement or pair — store and look up
- You need to group items by a property — use hash map with list values
- Any $O(n^2)$ problem involving pair comparisons — try hash map for O(n)

Interview Traps

□ **Trap**	Returning the wrong type — Two Sum wants indices, not values
□ **Trap**	Using a list instead of a set/dict for membership check — O(n) vs O(1)
□ **Trap**	Forgetting to handle the case where the same element is used twice

Edge Cases

- nums contains duplicates — handle with index tracking
- Multiple valid answers — return first found or all
- Target is 0 and array has zeros

Important Practice Problems

- LeetCode 1 — Two Sum
- LeetCode 49 — Group Anagrams
- LeetCode 347 — Top K Frequent Elements
- LeetCode 242 — Valid Anagram
- LeetCode 128 — Longest Consecutive Sequence

Real World Python Usage

□ **Real World**	Hash map counting powers inverted indexes in search engines (word → document list). Recommendation systems count user-item interactions in a hash map. Log parsers group events by type using frequency maps. Python's Counter is directly used in NLP for word frequency analysis.

3.5 Binary Search

The Problem Situation

You need to search in a sorted array, find the boundary of a condition, or determine the minimum/maximum value that satisfies a constraint. Binary search eliminates half the search space each step, giving O(log n) — versus O(n) linear scan.

Key Insight

□ **Insight**	At each step, check the middle element. If it matches your condition, record it as a candidate. Then decide whether the answer could be on the left half or right half and eliminate the other half. Repeat until the search space is empty.

Analogy

□ **Mental Model**	Looking up a word in a physical dictionary. You open to the middle. If your word comes before the middle word, throw away the right half. If it comes after, throw away the left half. Repeat. You find any word in 20 steps in a 1,000,000-word dictionary.

Basic Code Template

```
# Classic binary search — find exact target
def binary_search(nums, target):
    left, right = 0, len(nums) - 1
    while left <= right:
        mid = left + (right - left) // 2   # avoids overflow
        if nums[mid] == target:
            return mid
        elif nums[mid] < target:
            left  = mid + 1
        else:
            right = mid - 1
    return -1
```

✅**This Template Solves**	LeetCode 704 — Binary Search • LeetCode 33 — Search in Rotated Sorted Array

Alternative Template

```
# Left boundary — first position satisfying condition
def left_bound(nums, target):
    left, right = 0, len(nums)
    while left < right:
        mid = (left + right) // 2
        if nums[mid] < target:  left  = mid + 1
        else:                   right = mid
    return left  # first index where nums[i] >= target

# Search on answer space — minimum max subarray sum
def can_split(nums, k, mid):
    count, total = 1, 0
    for num in nums:
        if total + num > mid: count += 1; total = 0
        total += num
    return count <= k

def split_array(nums, k):
    left, right = max(nums), sum(nums)
    while left < right:
        mid = (left + right) // 2
        if can_split(nums, k, mid): right = mid
        else:                       left  = mid + 1
    return left
```

✅This Template Solves	LeetCode 278 — First Bad Version • LeetCode 410 — Split Array Largest Sum • LeetCode 875 — Koko Eating Bananas

Example Walkthrough

```
# Example: Search in Rotated Array — nums=[4,5,6,7,0,1,2], target=0
# left=0, right=6, mid=3 (val=7)
# Left half [4,5,6,7] is sorted: target(0) not in [4,7] → go right
# left=4, right=6, mid=5 (val=1)
# Right half [1,2] is sorted: target(0) not in [1,2] → go left
# left=4, right=4, mid=4 (val=0) → found at index 4
```

Time & Space Complexity

Case	Time	Space
Classic search	O(log n)	O(1)
Search on answer space	O(n log(max-min))	O(1)

Pattern Recognition Clues

- Keywords: sorted, find target, first/last occurrence, minimum/maximum satisfying condition
- If you can define a monotonic condition (true/false boundary), binary search works
- Search space is a range of numbers (not just array indices) — binary search on answer
- Any time you see O(n log n) hinted — a binary search inside a loop

Interview Traps

Trap	Using left <= right for boundary search — causes infinite loop. Use left < right for boundaries.
Trap	Calculating mid as (left+right)//2 — can overflow in other languages; use left+(right-left)//2
Trap	Forgetting to narrow the search space — not updating left or right leads to infinite loop

Edge Cases

- Single element array
- Target not present — return -1 or the insertion point
- Duplicates in array — affects leftmost/rightmost boundary
- Rotated sorted array — check which half is sorted first

Important Practice Problems

- LeetCode 704 — Binary Search
- LeetCode 33 — Search in Rotated Sorted Array
- LeetCode 153 — Find Minimum in Rotated Sorted Array
- LeetCode 875 — Koko Eating Bananas
- LeetCode 410 — Split Array Largest Sum

Real World Python Usage

Real World	Python's bisect module uses binary search internally. Database B-tree indexes use binary search at each node. Git bisect finds the first bad commit using binary search. NumPy's searchsorted() is binary search on sorted arrays. CDN load balancers use binary search to select servers by sorted capacity.

3.6 Monotonic Stack

The Problem Situation

You need to find the next greater element, next smaller element, previous greater element, or the largest rectangle/area for each element. Brute force compares each element with every other — $O(n^2)$. A monotonic stack processes each element at most twice (once pushed, once popped) giving O(n).

Key Insight

Insight	Maintain a stack that is always sorted (increasing or decreasing). When a new element breaks the sorted order, pop elements from the stack — each popped element has found its answer (the current element is its next greater/smaller). The key insight: each element is pushed once and popped at most once, so the total work is O(n).

Analogy

Mental Model	A line of people waiting to see who is taller than them. Each person looks back. If they are taller than the person behind them, that person behind has found their 'next taller person' (the current one) and steps out of line. The line always stays in height order.

Basic Code Template

```
# Next Greater Element — monotonic decreasing stack
def next_greater(nums):
    n      = len(nums)
    result = [-1] * n
    stack  = []   # stores indices
    for i in range(n):
        # while current element breaks the decreasing order
        while stack and nums[stack[-1]] < nums[i]:
            idx = stack.pop()
            result[idx] = nums[i]   # nums[i] is the next greater
        stack.append(i)
    return result
```

✓This Template Solves	LeetCode 496 — Next Greater Element I • LeetCode 739 — Daily Temperatures • LeetCode 503 — Next Greater Element II

Alternative Template

```
# Largest Rectangle in Histogram — monotonic increasing stack
def largest_rectangle(heights):
```

```
    stack    = []   # increasing stack of indices
    max_area = 0
    heights  = heights + [0]   # sentinel to flush stack at end
    for i, h in enumerate(heights):
        start = i
        while stack and heights[stack[-1]] > h:
            idx      = stack.pop()
            width    = i - (stack[-1] if stack else -1) - 1
            max area = max(max area, heights[idx] * width)
            start    = idx
        stack.append(start)
    return max_area
```

✅This Template Solves	LeetCode 84 — Largest Rectangle in Histogram • LeetCode 85 — Maximal Rectangle

Example Walkthrough

```
# Example: Daily Temperatures — [73,74,75,71,69,72,76,73]
# i=0 (73): stack=[]   → push 0.     stack=[0]
# i=1 (74): 74>73 → pop 0, result[0]=1. push 1. stack=[1]
# i=2 (75): 75>74 → pop 1, result[1]=1. push 2. stack=[2]
# i=3 (71): 71<75 → push 3.  stack=[2,3]
# i=4 (69): 69<71 → push 4.  stack=[2,3,4]
# i=5 (72): 72>69 → pop 4, result[4]=1
#            72>71 → pop 3, result[3]=2. push 5. stack=[2,5]
# i=6 (76): 76>72 → pop 5, result[5]=1
#            76>75 → pop 2, result[2]=4. push 6. stack=[6]
# Result: [1,1,4,2,1,1,0,0]
```

Time & Space Complexity

Case	Time	Space
All patterns	O(n)	O(n)
Each element pushed and popped once		

Pattern Recognition Clues

- Keywords: next greater, next smaller, previous greater, previous smaller, span, histogram
- You need to find the nearest element that is larger or smaller
- Area or span problems — largest rectangle, maximal rectangle
- Problem where each element 'waits' for something — use a stack to store waiting elements

Interview Traps

□ **Trap**	Storing values in stack instead of indices — you often need the index to compute distances/widths
□ **Trap**	Choosing wrong monotonic direction — decreasing stack for next greater, increasing for next smaller
□ **Trap**	Forgetting to handle remaining elements in stack after loop ends

Edge Cases

- Array with all elements equal
- Strictly increasing or decreasing array
- Single element array
- Circular array — process array twice (LeetCode 503)

Important Practice Problems

- LeetCode 739 — Daily Temperatures
- LeetCode 496 — Next Greater Element I
- LeetCode 84 — Largest Rectangle in Histogram
- LeetCode 85 — Maximal Rectangle
- LeetCode 42 — Trapping Rain Water

Real World Python Usage

□ **Real World**	Stock span calculations in financial systems use monotonic stacks. Compilers use monotonic stack-like structures for operator precedence parsing. Image processing computes local maxima using a monotonic approach. Python AST evaluation uses stack-based expression parsing.

3.7 Monotonic Queue

The Problem Situation

You need the maximum or minimum in a sliding window of size k as the window moves across the array. Naively recalculating the max for each window is O(nk). A monotonic deque maintains the maximum at the front in O(1), giving O(n) total.

Key Insight

Insight	Maintain a deque of indices in decreasing order of their values. When the window slides right, remove the front if it is outside the window. Remove all elements from the back that are smaller than the new element (they can never be the max while the new element is in the window). The front of the deque is always the maximum of the current window.

Analogy

Mental Model	A queue of people ranked by height. When a taller person joins at the back, all shorter people already in the queue behind them are useless — they will never be the tallest while the taller person is present. So shorter people leave when a taller person arrives.

Basic Code Template

```
# Sliding window maximum — monotonic decreasing deque
from collections import deque

def max_sliding_window(nums, k):
    dq     = deque()   # stores indices, values decreasing
    result = []
    for i, num in enumerate(nums):
        # remove indices outside the window
        while dq and dq[0] < i - k + 1:
            dq.popleft()
        # remove smaller elements from back — they are useless
        while dq and nums[dq[-1]] < num:
            dq.pop()
        dq.append(i)
        # window is fully formed
        if i >= k - 1:
            result.append(nums[dq[0]])
    return result
```

✓This Template Solves	LeetCode 239 — Sliding Window Maximum • LeetCode 1438 — Longest Subarray With Absolute Diff <= Limit

Example Walkthrough

```
# Example: nums=[1,3,-1,-3,5,3,6,7], k=3
# i=0 (1):  dq=[0]
# i=1 (3):  3>1 → pop 0, dq=[1]
# i=2 (-1): -1<3 → dq=[1,2],  window full → result=[nums[1]]=[ 3]
```

```
# i=3 (-3): -3<-1 → dq=[1,2,3], result=[3, 3]
# i=4 (5):  5>-3→pop3, 5>-1→pop2, 5>3→pop1, dq=[4], result=[3,3,5]
# i=5 (3):  3<5 → dq=[4,5], result=[3,3,5,5]
# i=6 (6):  6>3→pop5, 6>5→pop4, dq=[6], result=[3,3,5,5,6]
# i=7 (7):  7>6→pop6, dq=[7], result=[3,3,5,5,6,7]
```

Time & Space Complexity

Case	Time	Space
All cases	O(n)	O(k)
Each element enqueued and dequeued once		

Pattern Recognition Clues

- Keywords: sliding window maximum, sliding window minimum, window of size k
- You need max/min in every window of fixed size as it slides — monotonic queue gives O(n)
- Often combined with other patterns — sliding window + monotonic queue

Interview Traps

□ **Trap**	Storing values instead of indices — you need indices to check if elements are outside the window
□ **Trap**	Wrong removal condition: remove front when dq[0] < i - k + 1 (strictly less)
□ **Trap**	Confusing with monotonic stack — queue removes from both ends, stack only from top

Edge Cases

- k = 1 — every element is its own window maximum
- k = len(nums) — answer is the global maximum
- All elements equal

Important Practice Problems

- LeetCode 239 — Sliding Window Maximum
- LeetCode 1438 — Longest Continuous Subarray With Absolute Diff Less Than or Equal to Limit

- LeetCode 862 — Shortest Subarray with Sum at Least K

Real World Python Usage

Real World	Real-time sensor data monitoring uses sliding window max to detect anomalies. Video encoding tracks maximum frame complexity in a window. Trading systems compute rolling max/min for technical indicators. Python pandas .rolling().max() uses an efficient sliding window internally.

3.8 Fast and Slow Pointers (Floyd's Algorithm)

The Problem Situation

You are working with a linked list and need to detect a cycle, find the middle node, or find the start of a cycle. Two pointers moving at different speeds will eventually meet if a cycle exists — or the fast pointer will reach the end if no cycle exists.

Key Insight

Insight	Move the slow pointer one step at a time and the fast pointer two steps at a time. If there is a cycle, fast will lap slow and they will meet. If there is no cycle, fast reaches None first. To find the middle: when fast reaches the end, slow is at the middle.

Analogy

Mental Model	Two runners on a circular track. One runs at double the speed. If the track is circular (has a cycle), the faster runner will eventually lap the slower runner — they will meet. If the track has an endpoint, the faster runner just finishes first.

Basic Code Template

```
# Cycle detection — Floyd's Tortoise and Hare
def has_cycle(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
        if slow == fast:
            return True
    return False
```

✅This Template Solves	LeetCode 141 — Linked List Cycle • LeetCode 142 — Linked List Cycle II

Alternative Template

```
# Find middle of linked list
def find_middle(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
    return slow   # slow is at the middle

# Find start of cycle
def detect_cycle(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
        if slow == fast:
            # reset slow to head, keep fast at meeting point
            slow = head
            while slow != fast:
                slow = slow.next
                fast = fast.next
            return slow   # start of cycle
    return None
```

✅This Template Solves	LeetCode 142 — Linked List Cycle II • LeetCode 876 — Middle of the Linked List • LeetCode 234 — Palindrome Linked List

Example Walkthrough

```
# Example: has_cycle — 1→2→3→4→2 (cycle at node 2)
# Step 1: slow=2, fast=3
# Step 2: slow=3, fast=2  (fast wraps through cycle)
# Step 3: slow=4, fast=4  → MEET → cycle detected

# Example: find_middle — 1→2→3→4→5
# Start: slow=1, fast=1
# Step 1: slow=2, fast=3
# Step 2: slow=3, fast=5
# fast.next is None → stop. slow=3 is the middle
```

Time & Space Complexity

Case	Time	Space

Cycle detection	O(n)	O(1)
Find middle	O(n)	O(1)
Find cycle start	O(n)	O(1)

Pattern Recognition Clues

- Keywords: linked list cycle, middle of list, palindrome linked list, happy number
- Problem is on a linked list and requires O(1) space — fast/slow pointers
- Any problem where two things move at different rates in a sequence
- Happy number problem — same pattern applied to number sequences

Interview Traps

Trap	Checking slow == fast before the first move — they start at the same node
Trap	Not checking fast and fast.next before advancing — causes NoneType AttributeError
Trap	Returning slow directly for even-length list middle — returns second of two middle nodes

Edge Cases

- Empty list or single node
- List with exactly two nodes
- Cycle starting at head
- Even vs odd length list for middle finding

Important Practice Problems

- LeetCode 141 — Linked List Cycle
- LeetCode 142 — Linked List Cycle II
- LeetCode 876 — Middle of the Linked List
- LeetCode 234 — Palindrome Linked List
- LeetCode 202 — Happy Number

Real World Python Usage

Real World	Operating system deadlock detection algorithms use cycle detection in resource allocation graphs. Memory leak detectors use pointer-chasing to find circular

	references. Rho algorithm (Pollard's) for integer factorisation uses the fast/slow pointer concept. Python's gc module detects reference cycles using graph-based cycle detection.

3.9 Depth-First Search (DFS)

The Problem Situation

You need to explore all paths in a tree or graph, check connectivity, find a path, or compute a property that depends on subtree results. DFS goes as deep as possible down one path before backtracking. It is the foundation of tree traversal, graph exploration, and backtracking.

Key Insight

Insight	Use recursion (which uses the call stack) or an explicit stack. Visit a node, mark it, then recurse into each unvisited neighbour. The recursive structure naturally handles the backtracking — when a recursive call returns, you are automatically back at the parent node.

Analogy

Mental Model	Exploring a maze by always turning left. You go as deep as possible down one corridor. When you hit a dead end, you backtrack to the last junction and try the next corridor. You always explore one complete path before trying alternatives.

Basic Code Template

```
# DFS on a tree — recursive
def dfs_tree(root):
    if not root:
        return
    # Pre-order: process node BEFORE children
    print(root.val)
    dfs_tree(root.left)
    dfs_tree(root.right)
    # Post-order: process node AFTER children
    # (move print to after the recursive calls)
```

✓This Template Solves	LeetCode 104 — Maximum Depth of Binary Tree • LeetCode 112 — Path Sum • LeetCode 543 — Diameter of Binary Tree

Alternative Template

```
# DFS on a graph — with visited set
from collections import defaultdict

def dfs_graph(graph, node, visited):
    visited.add(node)
    for neighbour in graph[node]:
        if neighbour not in visited:
            dfs_graph(graph, neighbour, visited)

# DFS iterative — explicit stack (avoids recursion limit)
def dfs_iterative(graph, start):
    visited = set()
    stack   = [start]
    while stack:
        node = stack.pop()
        if node in visited:
            continue
        visited.add(node)
        for nb in graph[node]:
            if nb not in visited:
                stack.append(nb)
```

✓This Template Solves	LeetCode 200 — Number of Islands • LeetCode 133 — Clone Graph • LeetCode 695 — Max Area of Island

Example Walkthrough

```
# Example: Max Depth — tree: 3→[9, 20→[15,7]]
# dfs(3): 1 + max(dfs(9), dfs(20))
# dfs(9): 1 + max(dfs(None),dfs(None)) = 1+0 = 1
# dfs(20): 1 + max(dfs(15),dfs(7)) = 1+max(1,1) = 2
# dfs(3): 1 + max(1, 2) = 3
```

Time & Space Complexity

Case	Time	Space
Tree DFS	O(n)	O(h)
Graph DFS	O(V+E)	O(V)
h=tree height, V=vertices, E=edges		

Pattern Recognition Clues

- Keywords: all paths, connected components, explore, traverse, depth, subtree

- Tree problems almost always use DFS — traverse and compute something on each subtree
- Graph problems: count components, check connectivity, detect cycles
- When you need to process children before parent — use post-order DFS

Interview Traps

□ **Trap**	Forgetting the visited set on graphs — causes infinite loop on cycles
□ **Trap**	Using DFS instead of BFS for shortest path — DFS does not guarantee shortest path
□ **Trap**	Hitting Python recursion limit on deep trees — use iterative DFS with explicit stack

Edge Cases

- Empty graph or tree
- Disconnected graph — need outer loop to start DFS from every unvisited node
- Graph with self-loops
- Very deep recursion — switch to iterative

Important Practice Problems

- LeetCode 104 — Maximum Depth of Binary Tree
- LeetCode 200 — Number of Islands
- LeetCode 133 — Clone Graph
- LeetCode 543 — Diameter of Binary Tree
- LeetCode 695 — Max Area of Island

Real World Python Usage

□ **Real World**	Web crawlers use DFS to explore website link structures. Python's import system resolves module dependencies using DFS. Git's commit graph traversal uses DFS. File system traversal (os.walk) uses DFS. Compilers use DFS on the AST during code generation.

3.10 Breadth-First Search (BFS)

The Problem Situation

You need to find the shortest path in an unweighted graph, traverse a tree level by level, or find minimum steps to reach a state. BFS explores all nodes at distance 1 before distance 2, distance 2 before distance 3 — guaranteeing the shortest path in unweighted graphs.

Key Insight

Insight	Use a queue. Start by enqueuing the source. At each step, dequeue a node, process it, and enqueue all unvisited neighbours. Because a queue is FIFO, nodes are processed in order of their distance from the source — the first time you reach a node is via the shortest path.

Analogy

Mental Model	Dropping a stone in a pond. Ripples spread outward in circles — level 1 is all nodes one step from source, level 2 is two steps, and so on. BFS visits nodes exactly in this ripple order.

Basic Code Template

```
# BFS — shortest path in unweighted graph
from collections import deque

def bfs(graph, start, target):
    visited = set([start])
    queue   = deque([(start, 0)])   # (node, distance)
    while queue:
        node, dist = queue.popleft()
        if node == target:
            return dist
        for nb in graph[node]:
            if nb not in visited:
                visited.add(nb)
                queue.append((nb, dist + 1))
    return -1   # not reachable
```

This Template Solves	LeetCode 102 — Binary Tree Level Order • LeetCode 994 — Rotting Oranges • LeetCode 127 — Word Ladder

Alternative Template

```
# Multi-source BFS — multiple starting nodes simultaneously
def multi_source_bfs(grid):
    rows, cols = len(grid), len(grid[0])
    queue = deque()
    # enqueue all sources at the start
```

```
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == 1:   # source condition
                queue.append((r, c, 0))
    visited = set((r,c) for r,c,_ in queue)
    dirs = [(0,1),(0,-1),(1,0),(-1,0)]
    while queue:
        r, c, dist = queue.popleft()
        for dr, dc in dirs:
            nr, nc = r+dr, c+dc
            if 0<=nr<rows and 0<=nc<cols and (nr,nc) not in visited:
                visited.add((nr,nc))
                queue.append((nr, nc, dist+1))
```

✅This Template Solves	LeetCode 994 — Rotting Oranges • LeetCode 286 — Walls and Gates • LeetCode 542 — 01 Matrix

Example Walkthrough

```
# Example: BFS level-order tree — [3,9,20,null,null,15,7]
# Queue: [3]
# Dequeue 3 → enqueue 9,20. Level=[3]
# Queue: [9,20]
# Dequeue 9 → no children. Dequeue 20 → enqueue 15,7. Level=[9,20]
# Queue: [15,7]
# Dequeue 15,7 → no children. Level=[15,7]
# Result: [[3],[9,20],[15,7]]
```

Time & Space Complexity

Case	Time	Space
BFS traversal	O(V+E)	O(V)
Level order tree	O(n)	O(w) max width
Multi-source BFS	O(V+E)	O(V)

Pattern Recognition Clues

- Keywords: shortest path, minimum steps, level order, nearest, fewest moves
- Unweighted graph and you need shortest path — always BFS, never DFS
- Tree level-by-level processing — BFS with level tracking
- Multiple sources spreading simultaneously — multi-source BFS

Interview Traps

□ **Trap**	Using a list instead of deque for the queue — list.pop(0) is O(n), deque.popleft() is O(1)
□ **Trap**	Adding nodes to visited when dequeuing instead of enqueuing — causes duplicate processing
□ **Trap**	Forgetting to mark visited when enqueuing — same node can be added to queue multiple times

Edge Cases

- Disconnected graph — BFS from one node does not reach all nodes
- Target not reachable — return -1 or infinity
- Grid BFS — check bounds before enqueuing
- Level tracking — use len(queue) at start of each level loop

Important Practice Problems

- LeetCode 102 — Binary Tree Level Order Traversal
- LeetCode 994 — Rotting Oranges
- LeetCode 127 — Word Ladder
- LeetCode 542 — 01 Matrix
- LeetCode 286 — Walls and Gates

Real World Python Usage

□ **Real World**	Google Maps uses BFS for finding nearby places. Social networks use BFS for friend-of-friend suggestions. Web crawlers use BFS for breadth-first URL discovery. Netflix recommendation engine uses BFS-like expansion on its content graph. Python's importlib uses BFS-like resolution for circular import detection.

Pattern Comparison — Section 3A

Pattern	Best For	Core Idea	Key Data Structure
Two Pointers	Sorted arrays, pairs, partitioning	Move left/right toward each other	Array indices

Sliding Window	Contiguous subarray/substring	Expand right, shrink left	Hash map / Counter
Prefix Sum	Range sum, subarray sum = k	Precompute running total	Array + Hash map
Hash Map Count	Frequency, pairs, grouping	Store and look up complement	dict / Counter
Binary Search	Sorted array, answer space	Eliminate half each step	Array indices
Monotonic Stack	Next greater/smaller, histogram	Stack stays sorted, pop when broken	Stack (list)
Monotonic Queue	Sliding window max/min	Deque stays sorted, pop stale elements	deque
Fast/Slow Ptr	Cycle detection, middle of list	Two speeds, meet if cycle	Linked list pointers
DFS	All paths, subtree computation	Go deep, backtrack	Recursion / Stack
BFS	Shortest path, level order	Process level by level	deque (queue)

End of Section 3A. Continue with Section 3B — Patterns 3.11 to 3.20.